

PROGRAMACIÓN II

TERCERA ENTREGA DEL TRABAJO PRÁCTICO

Integrantes del grupo:

Grauberger, Federico

Longo Parra, Matias

Docentes:

Juárez, Ángel

Soria, Lucia

Comisión: 2

Cuatrimestre - Año: 2 - 2025

1. Enunciado del problema

NUEVOS REQUERIMIENTOS:

Partiendo de la solución propuesta en la primera parte, agregar los siguientes requerimientos.

La empresa ha decidido tener distintos tipos de contratación. Los empleados contratados que ya tenía anteriormente y los empleados de planta permanente, de los cuales se cuenta con los mismos datos salvo el valor hora, que no tienen; estos empleados tienen un valor por día de trabajo, y una categoría que puede ser “INICIAL”, “TÉCNICO” o “EXPERTO” (información que no será utilizada, pero es relevante para la empresa). Además, en el caso de las tareas de medio día, se considera que trabajan el día completo. Cuando el proyecto esté finalizado los empleados de planta que no hayan tenido retraso, recibirán un adicional del 2% sobre el pago correspondiente a las tareas realizadas por dicho proyecto. Este adicional es considerado como costo del proyecto inicialmente, es decir se toma en cuenta para el costo total al inicio del proyecto, en el caso de no cumplir con los plazos, no se abonará el adicional al empleado.

Será necesario saber que empleados tuvieron retrasos en las tareas asignadas y que cantidad de veces ocurrió. Esto debe realizarse para cualquier tipo de empleado.

La asignación de empleados a una tarea no discrimina entre los distintos tipos de empleados. En algún caso la empresa también quiere tener la posibilidad de tomar en cuenta la cantidad de retrasos de los empleados. Es necesario contar con dos tipos de asignación de empleado, una donde se asigna el primero libre que se encuentra y otra donde debería primero asignarse los que no tuvieron ningún retraso y en caso de no haber ningún empleado sin retrasos, seleccionar los que menos retrasos tuvieron.

A los proyectos en curso se les pueden agregar tareas, en ese caso se puede modificar la fecha de finalización prevista y la fecha real, ambas aumentan la cantidad de días que implica la nueva tarea. Las nuevas tareas se crean sin empleado asignado, por lo que será necesario asignar un empleado.

Las tareas tendrán estados, es decir pueden estar terminadas o no. Si la tarea está terminada, el empleado será liberado, pero debe quedar en el historial como responsable.

Una vez finalizado el proyecto, los empleados son liberados, es decir ya no quedan vinculados a la tarea. Es necesario que se mantenga un historial de empleados asignados a cada proyecto.

Es importante saber que la empresa puede contratar nuevas clases de empleados en el futuro. Esto debe impactar lo menos posible en el diseño de la solución.

Se agregan algunos requerimientos.

El sistema solicitado deberá permitir.

1. Registrar un nuevo empleado.
2. Registrar un nuevo proyecto.
3. Asignar un empleado a una tarea.
4. Asignar al empleado con menos retraso a una tarea.
5. Registrar retraso en una tarea
6. Agregar tarea
7. Establecer tarea como finalizada
8. Actualizar el estado del proyecto como finalizado.
9. Reasignar empleados de ser necesario (se debe poder cambiar un empleado asignado por otro) en $O(1)$
10. Reasignar empleado de forma eficiente (buscar el que menos atrasos tenga)
11. Informar rápidamente el costo final de un proyecto finalizado o no. $O(1)$
12. Informar los proyectos finalizados.
13. Informar los proyectos pendientes.
14. Informar los proyectos activos.
15. Informar empleados no asignados
16. Consultar rápidamente si un proyecto está finalizado o no.

17. Consultar si un empleado tuvo tareas con retraso.

18. Consultar empleados asignados a un proyecto.

2. TADs propuestos

Definimos que para poder cumplir con los requerimientos del problema, serán necesarios los siguientes tads:

- HomeSolution (principal)
- Proyecto
- Cliente
- Empleado
- EmpleadoPermanente
- Tarea

3. Modelado de cada TAD - Interface y estructuras de datos propuestas

HomeSolution

Datos:

Dicc. clientes // Dicc. para acceder rápidamente a los clientes mediante su email.

Dicc. empleados // Dicc. para buscar empleados de manera eficiente conociendo su número de legajo para reasignar en $O(1)$.

Dicc. proyectos // Dicc. para acceder de manera eficiente a los proyectos y sus datos mediante su número de identificación.

Operaciones:

void registrarEmpleado(String nombre, String legajo, int valorHora)

void registrarProyecto(Integer codigo, Cliente cliente, String dirección, List tareas, Fecha fechaInicio, Fecha fechaEstimadaFin)

Empleado asignarEmpleadoATarea(Tarea tarea)

Empleado asignarEmpleadoConPocoRetrasoATarea(Tarea tarea)

`void registrarRetrasoTarea(String tituloTarea, int tiempoRetraso, Integer codProyecto) // Registra un retraso en días para una tarea de un proyecto. Lanza excepción si tiempoRetraso es <= 0.`

`void actualizarEstadoProyectoAFinalizado(Integer codProyecto) // Marca un proyecto como finalizado cuando todas sus tareas están completadas. Lanza excepción si hay alguna tarea incompleta`

`void reasignarEmpleadoEnTarea(Tarea tarea, Empleado nuevoEmpleado)`

`int calcularCostoFinal(Integer codigoProyecto)`

`List<Proyecto> informarProyectosNoFinalizados()`

`List<Proyecto> informarProyectosNoIniciados()`

`boolean consultarFinProyecto(Integer código) // devuelve true si el proyecto consultado está finalizado y false en caso contrario.`

`List<Empleado> consultarRetrasosEmpleados()`

`void agregarTareaAProyecto(Integer codProyecto, Tarea nuevaTarea)`

`void finalizarTarea(Tarea tarea)`

`List<Proyecto> informarProyectosFinalizados()`

`List<Empleado> informarEmpleadosNoAsignados()`

`List<Empleado> informarEmpleadosEnProyecto(Proyecto proyecto)`

Cliente

Datos:

String nombre

String teléfono

String email

Operaciones:

String obtenerNombre()

String obtenerEmail()

String obtenerTelefono()

Empleado (abstracto)

Datos:

Tarea tareaEnCurso // indica qué tarea se está realizando por el empleado, si está vacío quiere decir que está disponible.

String nombre

String legajo // Utilizamos String debido a que el legajo podría contener otros caracteres aparte de números.

int cantidadDeRetrasos

List<Tarea> tareasRealizadas // Utilizamos una lista ya que no es necesario acceder de forma rápida a las tareas que el empleado realizó.

double costo

Operaciones:

String obtenerLegajo()

boolean estaAsignado() // Devuelve si un empleado está asignado a alguna tarea.

void asignarTarea(Tarea t) // asigna la tarea que está realizando el empleado. Lanza excepción si ya hay una tarea asignada.

void liberarEmpleado() // “libera” al empleado de la tarea que esté realizando. Lanza excepción si no hay ninguna tarea asignada.

void incrementarRetrasos()

void eliminarTarea() // elimina una tarea de la lista de tareas si el empleado fue reasignado.

List <Tarea> obtenerTareas() // Devuelve la lista de tareas que realizó el empleado

double calcularCosto() // método abstracto

EmpleadoPermanente

Datos:

```
// Hereda de Empleado
```

```
double valorDia
```

```
String categoria
```

Operaciones:

```
// Hereda de Empleado
```

```
double calcularCosto()
```

EmpleadoSimple

Datos:

```
// Hereda de Empleado
```

```
double valorHora
```

Operaciones:

```
// Hereda de Empleado
```

```
double calcularCosto()
```

Proyecto

Datos:

Integer id // Decidimos que en los proyectos los ID seran números

Cliente cliente

String direccionVivienda

List<Tarea> tareas

Fecha fechaInicio // Utilizamos fecha (de java) para poder manejar de mejor manera las fechas y aprovechar las funciones que la clase incluye

Fecha fechaEstimadaFin

Fecha fechaRealFin

double costoFinal

String estadoProyecto

Dicc <Empleado> empleadosEnProyecto

Operaciones:

double calcularCosto()

Fecha calcularFechaFin()

String verificarEstadoProyecto()

Fecha modificarFechaRealFin (Fecha fecha)

Double obtenerCosto()

void finalizarProyecto() // actualiza estado y calcula el costo. Lanza excepción si alguna tarea no está finalizada.

void agregarTarea(String tarea)


```
list <Empleado> listarEmpleadosEnProyecto()
```

Tarea

Datos:

String titulo

String descripción

double tiempoNecesario // Usamos double ya que medimos el tiempo en días y admite 0.5.

int tiempoRetraso // tiempo en días acumulados de retraso.

Empleado empleadoResponsable // Empleado que tiene asignada la tarea.

double costo

boolean finalizado //estado de la tarea

Operaciones:

void asignarEmpleado(Empleado empleado) // asigna un empleado a la tarea.
Lanza excepcion si el empleado tiene otra tarea asignada.

double calcularCosto() // calcula el costo de la tarea usando el valor del empleado y la duración final de la tarea.

void eliminarEmpleado() // elimina el empleado en caso de reasignacion. Lanza excepcion si empleado = null.

boolean tieneEmpleadoAsignado() // devuelve true si empleado != null, devuelve false en caso contrario.

void registrarRetraso(int tiempo) // registra un retraso en días. Lanza excepcion si es < 0.

void modificarTiempoNecesario(int tiempo) // cambia el tiempo esperado en base al retraso.

int tiempoNecesarioFinal() // devuelve el tiempo que llevó realizar la tarea sumando sus retrasos.

```
void finalizarTarea()
```

4. IREP

HomeSolution:

- * clientes no puede ser null. Todos los clientes en el HashMap deben tener un email que coincida con la clave que los almacena (String).
- * empleados no puede ser null. Todos los empleados deben tener un legajo que coincida con la clave que los almacena (Integer).
- * proyectos no puede ser null. Todos los proyectos deben tener un código que coincida con la clave que los almacena (Integer).

Proyecto:

- * codigo no puede ser null.
- * estado debe ser uno de los valores definidos en la clase "Estado" (PENDIENTE, ACTIVO, FINALIZADO).
- * cliente no puede ser null.
- * tareas (HashMap) no puede ser null. Todos las claves (String) deben coincidir con el título de la tarea correspondiente.
- * empleadosAsignados (HashMap) no puede ser null.
- * fechaInicio, this.fechaEstimadaFin y this.fechaRealFin no pueden ser null y deben ser un String con formato "yyyy-MM-dd".

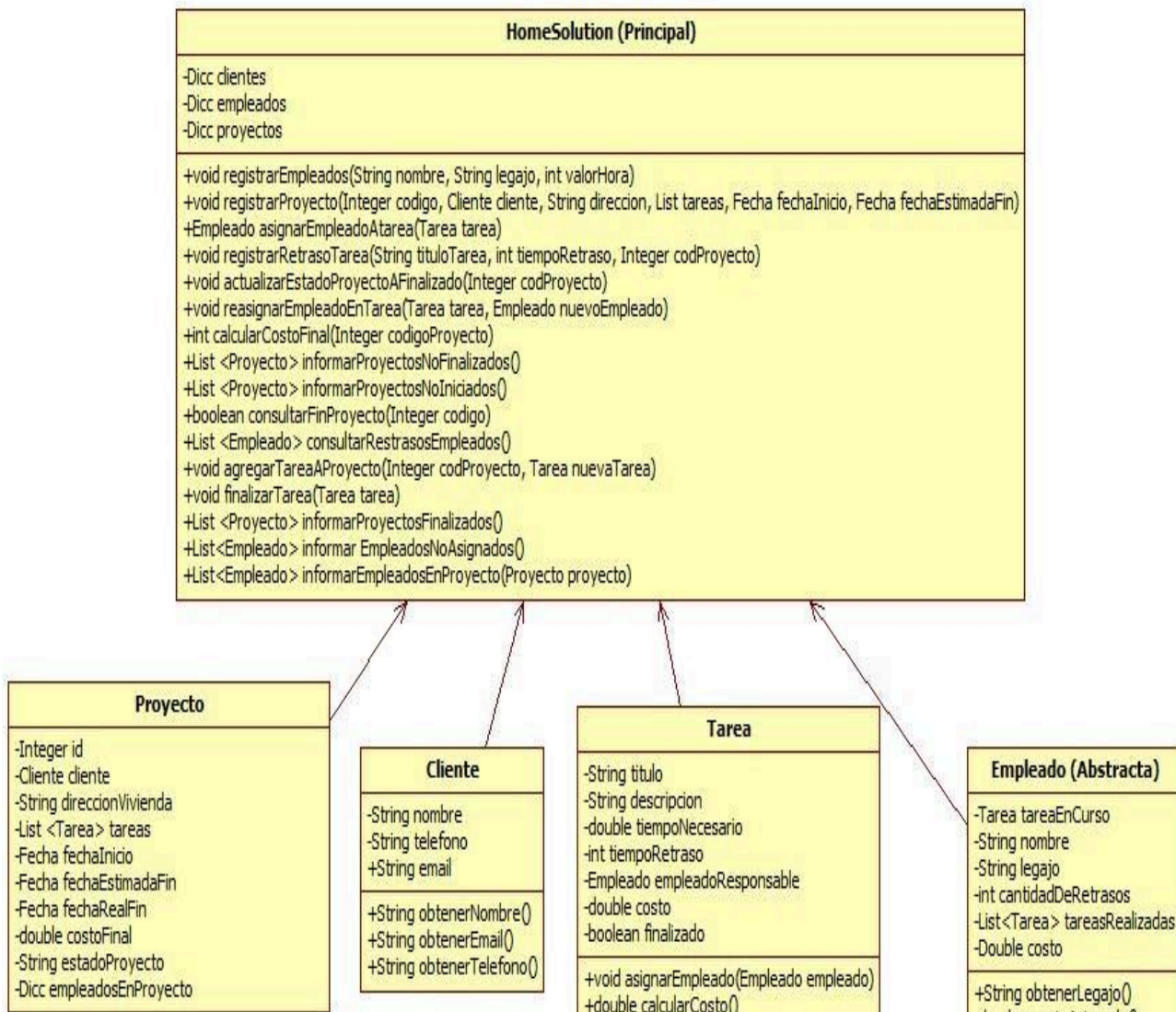
Empleado (Abstracta):

- * legajo no puede ser null y debe ser único (gestionado por "proximoLegajo").
- * nombre no puede ser null ni vacío.
- * cantidadDeRetrasos ≥ 0 .
- * tareaEnCurso es null solo si el empleado está disponible (no asignado).

Tarea:

- * titulo no puede ser null ni vacío.
 - * diasNecesarios debe ser ≥ 0.5 .
 - * retrasoAcum debe ser ≥ 0 .
 - * estado debe ser uno de los valores definidos en "Estado" ("FINALIZADO", "ACTIVO", "PENDIENTE").
 - * responsable (Empleado) es null solo si la tarea no está asignada. Si no es null, responsable.estaAsignado() debe ser true.
- Cliente:
- * email no puede ser null ni vacío, y debe ser la clave usada en el HashMap "clientes" de HomeSolution.
 - * nombre no puede ser null ni vacío.

Diagrama:



Desarrollo:

En el trabajo práctico entregado, se utilizaron los conceptos de herencia, polimorfismo, sobreescritura, sobrecarga y una clase abstracta. A continuación se detallan los momentos en que se usaron dichos conceptos.

Herencia y abstracción:

La herencia se utilizó para englobar los tipos de Empleados en una clase padre llamada Empleado, está a su vez es una clase abstracta, por lo que se incluyeron métodos abstractos, como el método calcularCosto(double cantDias).

Clase Empleado:

```
public abstract double calcularCosto(double cantDias);
```

Clase EmpleadoDePlanta:

```
@Override
public double calcularCosto(double cantDias) {
    // Medio día se redondea a 1 día completo
    double diasACobrar = Math.ceil(cantDias); // Math.ceil() redondea pa

    double costoBase = this.valorPorDia * diasACobrar;

    // El bonus del 2% se aplica si ESE empleado no tiene retrasos
    if (this.consultarCantidadRetrasos() == 0) {
        return costoBase * 1.02; // 2% adicional
    } else {
        return costoBase;
    }
}
```

Sobrescritura e Interfaces:

Para implementar los métodos abstractos mencionados anteriormente, se necesitó utilizar la sobreescritura en cada clase hija de la clase abstracta, por otro lado, también se sobreescriben los métodos que corresponden a las interfaces IHomeSolution y IProyecto.

Sobrecarga:

La sobrecarga se utilizó en los métodos `calcularCosto()` y `registrarEmpleado()` ya que dependiendo del tipo de empleado es diferente el tipo de datos que recibe por parámetros y así mismo, diferente será la resolución del cálculo del costo.

Clase `HomeSolution`:

```
@Override
public void registrarEmpleado(String nombre, double valor) throws IllegalArgumentException {
    if ((nombre == null || nombre.length() == 0) || valor < 0) {
        throw new IllegalArgumentException("Nombre o valor inválido");
    }

    Empleado empleadoAux = new EmpleadoSimple(nombre, valor);
    this.empleados.put(empleadoAux.consultarLegajo(), empleadoAux);
}

@Override
public void registrarEmpleado(String nombre, double valor, String categoria) throws IllegalArgumentException {
    boolean categoriaValida = categoria.equals("INICIAL") || categoria.equals("TECNICO") || categoria.equals("EXPERTO");
    if ((nombre == null || nombre.length() == 0) || valor < 0 || !(categoriaValida)) {
        throw new IllegalArgumentException("Nombre, valor o categoria inválido");
    }

    Empleado empleadoAux = new EmpleadoDePlanta(nombre, valor, categoria);
    this.empleados.put(empleadoAux.consultarLegajo(), empleadoAux);
}
```

Polimorfismo:

Queda evidenciado el buen uso del polimorfismo ya que no se requirió “castear” en ningún lugar del código. Sobre todo en los métodos que asignan y reasignan responsables en tareas. En ambos casos, la clase `Proyecto` recibe como al empleado como objeto “Empleado” (abstracto), sin importar si este es un empleado de planta o un empleado contratado (`EmpleadoSimple`), haciendo que con el mismo método cada uno se comporte como debería.

Herramientas de Tecnologías Java:

Iteradores y Foreach: Para el recorrido de las colecciones principales (`HashMap` de empleados, proyectos y tareas). Esta abstracción del iterador nos permite recorrer los valores de las colecciones de forma limpia y eficiente, como se observa en los métodos `"empleadoDisponible()"` o `"consultarCosto()"`.

StringBuilder y toString(): se utilizó `StringBuilder` en los métodos `toString()` de las clases `"HomeSolution"` y `"Proyecto"`. En lugar de concatenar muchos `String`.

Análisis de complejidad (Puntos 9 y 10):

- El punto 9 (Reasignar un empleado por otro) se resuelve en $O(1)$ ya que en ningún momento de todo el flujo del método se utiliza un bucle para recorrer. Sino que todas las instrucciones son constantes ($O(1)$) por ser asignaciones o consultas a variables.
- El punto 10 (Reasignar al que menos retrasos tenga) se resuelve en complejidad $O(n)$, siendo “n” el número total de empleados (los cuales hay que recorrer uno por uno para determinar cuál es el que tiene menos retrasos). Es imposible (con la implementación actual) resolver este punto en una menor complejidad, ya que implicaría verificar en cada registro de retraso cual es el empleado con menos retrasos (una especie de empleado del mes), por lo tanto, debido a nuestra implementación, el método `reasignarEmpleadoConMenosRetraso()` es $O(n)$ en el peor caso.